SAFEROUTE

Adaptive Disaster-Resilient Communication Network Simulator

Innovative Evaluation Practices

BCSE308P — Computer Networks Lab Project

Team	Team 11
Slot	F1
Team members	Abhinav Sharma (24BCT0116)
Faculty	Sreethar s
Institution	Vellore Institute of Technology, Vellore

Abstract

SafeRoute is an educational communication-network simulator designed to demonstrate how a network can respond when links or nodes become unavailable during a disaster. The simulation models routers, switches, and emergency-service endpoints such as hospitals, police stations, fire stations, and rescue centres. It supports packet injection, shortest-path routing using Dijkstra's algorithm, priority handling for emergency traffic, congestion scenarios, component failure and recovery, and a live dashboard for observing network state and performance metrics. The project uses a JavaScript simulation engine running on Node.js, an Express API, and a React-based frontend visualized with React Flow. SafeRoute is intended for learning and demonstration; it is not a real-world emergency communications platform or a packet-level network emulator.

1. Introduction

Communication networks are important during disasters because emergency organizations need to exchange information even when parts of the network are damaged or congested. A failed link can disconnect a destination from the rest of the network, while heavy traffic can delay important messages. SafeRoute provides a controlled virtual environment where these situations can be created, observed, and discussed without requiring physical networking equipment.

The project represents a network as nodes connected by links. Nodes can represent networking devices or service endpoints. Links have properties such as bandwidth and latency, and the simulator advances in discrete time steps. Users can send normal or emergency packets, trigger disaster presets, fail and recover components, and observe the effects in the dashboard.

2. Problem Statement

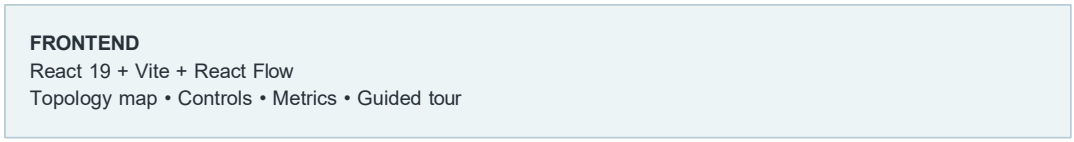
To develop an interactive simulator that demonstrates network routing, emergency packet prioritization, congestion, and recovery from component failures, and that makes the resulting topology changes and performance measurements visible to students through a web dashboard.

3. Objectives

- Model a network topology containing routers, switches, and emergency-service nodes.
- Route packets over available paths using Dijkstra's shortest-path algorithm.
- Give emergency packets preferential queue handling over normal traffic.
- Simulate link/node failures, congestion, disaster presets, and recovery.
- Display topology, packet activity, routing outcomes, and performance metrics.
- Provide a guided tour and automated tests to support demonstration and verification.

4. System Architecture

The system is divided into three main layers. The React frontend renders the network and sends user actions to the backend. The Express API validates requests and coordinates a background simulation clock through the SimulationManager. The engine contains the network model and simulation logic, including nodes, links, packets, queues, routing, failures, and metrics.



BACKEND API
Node.js + Express
Request validation • API routes • SimulationManager / clock

SIMULATION ENGINE
Node • Link • Packet • Network • Simulator
Routing • Queues • Failures • Recovery • Metrics

The backend exposes endpoints for topology and metrics retrieval, simulation start/pause/reset, normal and emergency traffic, node/link management, component failure/recovery, and disaster triggering. The dashboard is served from the built frontend directory when using the default single-port setup.

5. Technologies Used

Layer	Technology	Role
Simulation engine	JavaScript / Node.js	Models nodes, links, packets, routing, queues, failures, and metrics.
Backend API	Express	Provides HTTP endpoints and coordinates the simulation clock.
Frontend	React 19 / Vite	Provides interactive controls and dashboard views.
Topology visualization	React Flow	Displays nodes and links as an editable network graph.
Testing	Node.js built-in test runner	Runs unit and API tests.

6. Core Concepts and Algorithms

6.1 Graph representation

The topology is represented as a graph. Each node represents a device or service, and each link represents a connection. Links are undirected in this simulator and include bandwidth and latency attributes. Failed components are excluded from usable routing paths.

6.2 Dijkstra's shortest-path routing

Dijkstra's algorithm calculates a shortest route from a source to a destination across available links. When a link or node fails, the simulator can recompute a route using the remaining topology. If no valid route exists, a packet may be marked unreachable and dropped. The exact route cost and tie-breaking behaviour should be described according to the implementation in engine.js.

6.3 Packet queues and emergency priority

Packets are generated with a size and priority. Link bandwidth determines how long packet serialization takes, while latency represents propagation delay. Emergency traffic receives preferential queue handling, allowing it to move ahead of normal traffic in the modeled queue. This is a simplified priority mechanism and should not be interpreted as a complete implementation of a real QoS standard.

6.4 Congestion and metrics

When offered traffic competes for limited link capacity, queues can build up and packets can experience delay. The dashboard displays measures such as latency, throughput, packet loss, and link utilization. These metrics help compare network behaviour before, during, and after a failure or congestion event.

7. Functional Modules

• Network model: manages node and link creation, modification, removal, failure, and recovery.
• Packet and simulator logic: creates packets, advances simulation ticks, manages delivery status, and records outcomes.
• API layer: validates requests and exposes simulation operations through HTTP endpoints.
• Dashboard: visualizes the topology, packet activity, metric cards, utilization bars, and controls.
• Guided tour: runs nine predefined teaching scenarios, including basic routing, congestion, emergency traffic, link failure, network partition, recovery, and a cyclone preset.
• Test suite: includes unit and API tests. The project documentation reports 62 tests; run the current test suite and record the actual result before submission.

8. Typical Demonstration Procedure

1. Start the application using npm install followed by npm start from the project root.
2. Open the local dashboard at http://127.0.0.1:8000.
3. Use the guided tour to create a small network and send a normal packet.
4. Trigger congestion and compare the visible queue and link-utilization indicators.
5. Send an emergency packet and observe its priority handling.
6. Fail a link and observe whether traffic is rerouted.
7. Partition the network and observe packets that cannot reach their destination.
8. Recover the failed components and repeat the traffic test.
9. Reset the simulation and inspect the metrics and event log.

9. API Overview

Endpoint / method	Purpose
GET /topology	Returns nodes, links, queues, packets, routes, and failures.
GET /metrics	Returns current simulation performance metrics.
GET /events	Returns chronological events, optionally filtered by since/limit.
POST /simulation/start, /pause, /reset	Controls the simulation clock and resets state.
POST /traffic/normal, /traffic/emergency	Injects packets with the requested priority.
POST /nodes, /links; PATCH /nodes/:id, /links/:id; DELETE equivalents	Creates, updates, and removes topology components.
POST /nodes/:id/fail or recover; /links/:id/fail or recover	Changes node or link availability.
POST /disasters/trigger	Applies a configured disaster scenario.

10. Testing and Expected Results

The project includes a Node.js test suite covering engine and API behaviour. The supplied project description states that 62 tests are present. Before submission, run npm test and replace this statement with the actual observed test result. Do not claim a test passed unless the command was executed successfully.

Expected observations include: a reachable destination receives packets when a valid route exists; link failure causes route recalculation where an alternate path exists; a partitioned network makes some destinations unreachable; emergency packets receive preferential queue treatment; and the metrics change as traffic, failures, and recovery are applied.

11. Limitations

SafeRoute is a teaching simulator, not a real network emulator. It uses whole-second simulation ticks, undirected shared-bandwidth links, centralized route computation, and category-based disaster presets. It does not model real sockets, TCP retransmission, routing-protocol convergence, packet fragmentation, or physical devices. Simulation state is held in memory in one process and is not persistent. These simplifications keep the project approachable but mean that its results should not be treated as a prediction of a production network.

12. Future Enhancements

Possible future extensions include configurable routing metrics, more detailed QoS policies, packet retransmission models, routing convergence visualizations, exportable experiment results, persistence of simulation sessions, and comparison of multiple routing strategies. A future version could also include a reproducible set of quantitative experiments to compare baseline routing with failure-aware routing under the same traffic patterns.

13. Conclusion

SafeRoute brings several Computer Networks concepts together in an interactive environment. By combining graph-based routing, packet queues, emergency priority, failure and recovery controls, and a visual dashboard, it enables students to explore the effect of network disruption and the role of alternate paths. The project is best evaluated through a live demonstration supported by the source code, test output, screenshots, and a clear explanation of its simulation assumptions.

References and Project Resources

1. Node.js documentation: https://nodejs.org/docs/
2. Express documentation: https://expressjs.com/
3. React documentation: https://react.dev/
4. React Flow documentation: https://reactflow.dev/